Applying deterministic behavior into userspace OSes
1 message

IoT Mote from IoT's Newsletter <iotmote@substack.com> Mon, Oct 31, 2022 at 3:23 PM
Reply-To: IoT Mote from IoT's Newsletter
<reply+1coeu1&uc415&&02f480b53c07997c142f5ce26aa4e09c2663802d0ac9257efe259249c965349d@mg1.substack.com>
To: giovanni.lostumbo@gmail.com

Open in app or online

Applying deterministic behavior into userspace OSes

Why smartphones could be more like RTOSes, with some tradeoffs

IOT MOTE
OCT 31

SAVE

In a theoretical world, one is not constrained by conventions. Syncretism is "the attempted reconciliation or union of different or opposing principles, practices, or parties, as in philosophy or religion." RTOS

I will attempt to address the impossible, because maybe I just like being a difficult person. What if a "smartphone" used RTOS concepts and got rid of its userspace to make phone calls and send text messages? That is the syncretic application of RTOS into the smartphone. If you grew up in the area of landlines, you might recall that landlines could make a call halfway around the world in a determinant amount of time: https://www.quora.com/Why-is-talking-on-a-landline-phone-to-someone-on-the-other-side-of-the-world-instantaneous-without-any-lag-Did-it-used-to-be-this-way-or-have-landlines-been-digitized

IoT's Newsletter is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Subscribed

“It actually is the digital processing that introduces delays. On analog communication the only delay is the time the signal takes to travel. On digital transmission there can be delays in conversions from analog to digital, buffering in between etc. Of course they try to keep it to minimum.”

“The scheduler in a Real Time Operating System (RTOS) is designed to provide a predictable (normally described as *deterministic*) execution pattern. This is particularly of interest to embedded systems as embedded systems often have real time requirements.”

In a theoretical world, immediate applications are not a concern. Applications may be the goal, but I am interested in learning more concepts, rather than building a slightly different app that uses the same OS, and same kernel. This is where the basic sciences and applied sciences have a border. If you are in the latter group, you may not like this post, because a year from now, I may still not have made much progress in developing such an app. Fair warning.

This post is more an exploration of why smartphones behave the way they do, and to examine why alternatives are not sought. Obviously people buy new phones for additional features. But with added features, some of the original functionality becomes slower to access, due to the added complexity of a modern operating system. This is an irrefutable fact with decades of history.

Some solutions have been offered:

“**Modularity**

Another solution for feature creep is modularity. Power users who require more functionality can retrofit needed features by downloading software modules, plug-ins, add-ons (also known as add-ins) and custom themes to match their personal requirements.

Pruning

At some point, the cost of maintaining a particular subset of features might become prohibitive, and pruning can be used. A new product version could simply omit the extra features, or perhaps a transition period would be used,

where old features were deprecated before eventual removal from the system. If there are multiple variations of products, then some of them might be phased out of use. One major example is the Samsung Galaxy S6, released March 2015, of which significantly many software/menu features and also some hardware features were pruned. A “more functional” variation of it hasn't been released.[*citation needed*]”

To further examine why an RTOS would be beneficial for phone calls and texts, consider the emergency function of a phone. Why is it there? Obviously to make a phone call in an emergency situation. Would it be too unreasonable to say that the act of calling 911 is now a “mission critical” action? May I unequivocally compare a life-saving tool to a safety mechanism in an industrial control system?

https://en.wikipedia.org/wiki/Mission_critical#Real_time_and_mission_critical

Types of real time systems

- Hard real-time system shouldn't miss the specified time or can result in serious consequences. It is non-negotiable in timing and it is "wrong answer" if the deadline is missed. The example of hard real time system is airbags for cars.[26]
- Soft real-time system has more loose deadline. The system can handle the problems and functions normally even though the deadline is missed, but their functionality depends on fast-paced processes. **An example for soft-real time can be typing, which, if delayed, people will get annoyed, but answer still is correct.[27]**
- Non real-time system doesn't have certain or absolute deadlines. However, the throughput of the activity of performance can still be very essential.[28]

Differences

Real time is a software that if specific time is not met, it fails, but mission critical is a system if failed, will result in catastrophic consequences. Although they go hand-in-hand, since real time can be mission critical, they are not the same concepts. These two are often confused by many people, but they are different concepts, but associated with each other.[29][30]”

Ok, so typing has already been categorized/identified as a soft-real time system. Anyone who has ever used a Windows system is familiar with the Blue Screen of Death. Android phones have an added benefit of having a

capacitive touch screen which functions as little more than an unprivileged symbol of hope, whereas the CTRL+ALT+DEL feature is non existent. In its place, is a semi-reliable power button, which may or may not prompt a restart/power off button, provided the phone isn't overheating from using GPS for barely 30 minutes of driving. The perils of placing too much dependencies on a modern marvel. Make no illusion, phones that malfunction while driving can cause collisions. So it is no minor quibble that I am bringing this up. It is far better to invest in a dedicated GPS device that has no annoying popups for advertisements or notifications while driving. You can call this first world problems, but the reality is, smartphones are owned by 86% of the world. Thus is it an endemic problem, potentially affecting motorists around the world. I doubt I am the first to complain about this, but in 20 years of using smartphones, I have never read of any Slashdot article, CNET article or any article anywhere that has attempted to sell an feature phone because it can't be upgraded, and would not make much profit, or any. There lies the crux of writing. I write because I try to offer people alternatives. I try to educate others on what they have traded. With all new technologies comes a qualitative change in the way of life. A new phone is not just a new phone. It is an omnibus of features, both hardware and software, that a committee has voted on, to represent the most utilitarian solution that could be manufactured into a single product. It cannot represent a single solution for a single task. Gone are the days of the rotary phone. Technological convergence is the norm because of economies of scale. Thus, it is vital to to not place phones on a pedestal. They may accomplish many tasks, and may be able to multi task faster than many predecessors. But they cannot approach the simplicity of a hard-real time OS. Something that follows the scientific method.

What I am proposing is developing a communication device that is 99.999999% reliable, in that the input/output of the keys appear on the screen in a predictable manner, and are never queued by some non-essential process like a browser or an advertisement. Consider this quote from Nat Friedman:

"We know less than we think

- The replication crisis is not an aberration"

The replication crisis, is in reference to the ability (or inability) to reproduce scientific results. https://en.wikipedia.org/wiki/Replication_crisis

Applied to user experience (after all, computers are developed by scientists), repeatability of a process should be able to proceed within a defined window

of I/O. That is, if a kernel scheduler allows I/O to be received from a keyboard within a certain time frame, a hard RTOS, if designated solely for that purpose, would be able to reliably output the characters of that text within a certain time frame. This is the fundamental design philosophy of my proposal. It designates I/O first, rather than kernel housekeeping. In theory, the separation between kernel and user mode is more about isolating processes from other processes, rather than protecting the OS from getting corrupted from user input- in this sense- an exokernel could be designed as a more dedicated, low level application that has a revolving set of tasks that are “use it or lose it” that is, the RTOS can operate a set number of tasks in a serial process, but it cannot introduce variability in the pipeline of tasks- much like how RISC instruction sets are not Complex and variable. The OS operates by processing “dummy sectors” I/O inputs that get voided but perhaps sent with a null function to keep the machine running in a deterministic fashion. This is the core mechanism of how I envision a deterministic userspace OS to function. The terminology “userspace” is broad, because it implies user input- but I have opted to adopt the most limited interpretation of that meaning- a single task- while in theory it could be a single button as on a remote control- on or off- here a qwery keyboard treats any input as that “one” character input. The OS “merely” displays that input on the screen, which for the purposes of that app, may be later stored on a disk or RAM, so that it can later be sent to a mobile network or wifi router for the purpose of sending a text message. This is just one idea of how a userspace RTOS could be designed. It would not be a single app in the sense that the phone/hardware is only capable of storing one app, but its processor architecture would be such that only one app could be given the privileged mode while the user is interacting with the phone, expecting an output. In theory, this would require unifying the display driver bus latencies into the factored amount of time that the OS would be ready for another input. To some extent this is already well documented, and some OSes probably have a better grip on a text editor that is able to do this. The next step would be to catalogue and benchmark a number of OSes and programs that can meet an extremely low tolerance level, and thus be repeatable in a predictable manner. While the actual use case may not always be mission critical, it is important to be able to have the option to access a mission-critical mode of an OS/phone hardware so that from the moment it boots (and even right before it), it can reliably be interacted with in a pseudo-analog manner- that is- its responsiveness would be as if a light switch faced no invisible obstacles from being switched on or off. All the barriers would be user-driven. If the user is unable to reach the switch, or if the user does not apply enough pressure to the switch. A light switch does not have a delay (one that is not related to an LED circuit if the

power is running). In other words, The light will most likely turn on within 3 seconds. A smartphone can take anywhere between less than one second to hours or days to return to a blinking cursor.

Thus my emphasis on designing an OS to function like a mechanical typewriter is not just some writer's fantasy. It is purely out of the discovery that RTOSes are a thing, and deterministic RTOSes could be applied to user space apps to produce increased reliability of I/O. I can definitely understand the low priority of developing such a device. Most people have the patience to wait even a few milliseconds to make a call. This isn't really about that. It's not necessarily designed for regular use. But the fact that hard-real time OS have a technology that could be useful to an extreme scenario where a split second haptic input is the difference between life or death, then yeah, I have the patience to explore that realm.

It could be said I have even more patience to explore that field than one who is too bothered by every day needs. I don't blame others for that. But I have a different motivation.

Now, I haven't yet covered every kind of delay in an operating system. Some delays are not caused by other apps or processed. Some delays, or errors, are caused by cosmic rays: https://gizmodo.com/space-weather-can-alter-electronic-voting-machines-1792473286

This is one form of what is called a "soft-error" https://en.wikipedia.org/wiki/Soft_error

"In electronics and computing, a **soft error** is a type of error where a signal or datum is wrong. Errors may be caused by a defect, usually understood either to be a mistake in design or construction, or a broken component. A soft error is also a signal or datum which is wrong, but is not assumed to imply such a mistake or breakage. After observing a soft error, there is no implication that the system is any less reliable than before. One cause of soft errors is single event upsets from cosmic rays."

"A **single-event upset** (**SEU**), also known as a **single-event error** (**SEE**), is a change of state caused by one single ionizing particle (ions, electrons, photons...) striking a sensitive node in a micro-electronic device, such as in a microprocessor, semiconductor memory, or power transistors. The state change is a result of the free charge created by ionization in or close to an important node of a logic element (e.g. memory "bit"). The error in device output or operation caused as a result of the strike is called an SEU or a soft

error.

The SEU itself is not considered permanently damaging to the transistor's or circuits' functionality unlike the case of single-event latch-up (SEL), single-event gate rupture (SEGR), or single-event burnout (SEB). These are all examples of a general class of radiation effects in electronic devices called *single-event effects* (SEEs)."

After reading this, one may accept that even an innocent natural particle can contribute to what I referred to earlier about the scientific method and how Nat Friedman says the replication crisis is not an *aberration*. In computer science- whether a software *works*, and repeatability, is the standard measure of of its quality and adoption.

Software intrinsically needs disk space, or RAM, at the very least to run. When the disk space or RAM gets corrupted while running a program, whether it is intermittent, or a single event, is a contributing factor to the perception of aberrations. This also has applications (and uses implementations of) to ECC RAM https://en.wikipedia.org/wiki/Error_correction_code

Thus, errors can happen at any level of a computer system, but the fact that the errors can multiply when complexity in multiple component of that systems should be a cause of concern. It is not always possible to debug or determine the root cause of an error. Some cosmic rays may only manifest their error in retrospect. Some errors may manifest themselves instantly with a completely different operation, or lack thereof. Thus to fully examine software options in choosing a technology, whether it is a landline, a "dumbphone" or a "smartphone," it my critical view that userspace OSes should not be the only foundational kernel types considered when designing new phone hardware; rather, RTOSes are a part of computer science, and computer science is a part of all basic sciences, which is not limited to electronics, but physics math and chemistry.

Thus, to examine the statistical probability of aberrations in deterministic RTOSes should be a goal for userspace OS developers interested in more reliable phones. As mentioned earlier, no new technology is without tradeoffs- exploring concepts in analog and mechanical computers could have a renewed application to digital circuitry, radiation hardened devices withstanding. Some amount of aberration and SEEs is likely to be expected- but whether those aberrations become so noticeable as to render the technology unusable should be the focus of RTOS userspace developers.

IoT's Newsletter is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Subscribed

Thanks for subscribing to IoT's Newsletter. This post is public, so feel free to share it.

Share

LIKE COMMENT SHARE

© 2022 IoT Mote
548 Market Street PMB 72296, San Francisco, CA 94104
Unsubscribe

Get the app Start writing